

Swagger Petstore API Defect Catalog

Documented defects observed against `https://petstore.swagger.io/v2/`, captured by a Playwright API diagnostic that exercised every endpoint of every tag (`pet` , `store` , `user`) with a battery of probes covering happy path, validation errors, not-found ids, eventual consistency, idempotency, upsert semantics, auth-header enforcement, schema fidelity, and edge inputs (empty bodies, malformed JSON, wrong content-type, special characters, out-of-range ids, non-numeric ids).

The OpenAPI spec at <https://petstore.swagger.io/v2/swagger.json> (referred to below as **the spec**) is treated as the source of truth for expected behaviour.

How findings were collected

A single Playwright spec ran four `test()` blocks in parallel — one per tag plus a cross-cutting block — issuing **78 probes** total against the live public sandbox. For each probe the diagnostic captured: HTTP status, status text, `Content-Type` , response body (text + parsed JSON when applicable), elapsed ms, and any client-side error. Output was written to `test-results/swagger-petstore-diagnostic/<area>.json` (gitignored).

Specifically captured per area:

- **Pet (30 probes)** — `POST/PUT/GET/DELETE` on a fresh id, immediate read-after-write, eventual-consistency retry loop, non-existent / zero / negative / non-numeric ids, `findByStatus` for `available|pending|sold|banana|<missing>|<multi>` , `findByTags` , form-data `POST /pet/{id}` , multipart `uploadImage` , idempotent `DELETE`, with/without `api_key` , malformed JSON body, `text/plain` content-type, missing required fields, empty body.
- **Store (18 probes)** — `inventory` with and without bogus `api_key` , `POST /store/order` happy / empty / invalid status / id outside documented 1..10 range, `GET /store/order/{id}` for valid + 0 + 11 + 9_999_999 + -1 + non-numeric, idempotent `DELETE`.
- **User (25 probes)** — `POST /user` happy + duplicate + empty + special-character username, `GET /user/{username}` immediate + eventual-consistency loop, batch endpoints (`createWithArray` , `createWithList`) including empty-array, `PUT /user/{username}` update + upsert, `GET /user/login` for valid creds + wrong password + non-existent user + empty params, `GET /user/logout` , idempotent `DELETE`, non-existent `DELETE`.
- **Cross-cutting (5 probes)** — `DELETE /pet/{id}` with no `api_key` and with bogus `api_key` , schema-fidelity round-trip (`POST` then `GET`, deep-equality of sent vs returned bodies), error response `Content-Type` sniff, `OPTIONS /pet` CORS preflight.

Severity classification

- **Critical** — security or data-integrity defects: missing auth enforcement, accepting any credentials, leaking implementation details.
- **High** — validation entirely absent, leading to silent acceptance of malformed input or duplicate keys in returned data.
- **Medium** — status-code drift (e.g., 404 instead of documented 400), incorrect idempotency semantics.
- **Low** — minor surface deviations (empty body without `Content-Type` on some 404s).

Summary

ID	Area	Severity	One-liner
AUTH-1	pet	Critical	<code>DELETE /pet/{id}</code> accepts the request with no <code>api_key</code> header (spec says required)
AUTH-2	user	Critical	<code>GET /user/login</code> returns success for any credentials, including non-existent users and empty params
DISCLOSURE-1	pet	Critical	<code>GET /pet/{non-numeric-id}</code> 404 body leaks a Java <code>NumberFormatException</code> stack-trace fragment
VAL-1	pet	High	<code>POST /pet</code> with empty body returns 200 and id <code>Long.MAX_VALUE</code>
VAL-2	pet	High	<code>POST /pet</code> with malformed JSON returns 500 "something bad happened" instead of 400
VAL-3	pet	High	<code>GET /pet/findByStatus?status=banana</code> returns 200 with empty array (spec says 400)
VAL-4	pet	High	<code>GET /pet/findByStatus</code> with no <code>status</code> param returns 200 (spec says 400)
VAL-5	pet	High	<code>GET /pet/findByTags</code> with no <code>tags</code> param returns 200 (spec says 400)
VAL-6	store	High	<code>POST /store/order</code> accepts empty body, invalid status, and ids outside the documented 1..10 range
VAL-7	user	High	<code>POST /user</code> accepts empty body, duplicate username, and special characters (/ , * , !) in the username
VAL-8	user	Medium	<code>POST /user/createWithArray</code> and <code>createWithList</code> accept an empty array and return "ok"
VAL-9	pet	Medium	<code>POST /pet/{id}</code> form-data accepts an empty form

CODE-1	pet	Medium	GET /pet/{invalid-id} returns 404 for 0, -1, and non-numeric ids (spec says 400)
CODE-2	store	Medium	GET /store/order/{invalid-id} returns 404 for -1, non-numeric (spec says 400)
CODE-3	store	Medium	DELETE /store/order/{invalid-id} returns 404 for non-numeric ids (spec says 400)
SEM-1	pet	High	PUT /pet silently creates pets with non-existent ids (upsert; spec says 404)
SEM-2	user	High	PUT /user/{nonexistent} silently creates the user (upsert; spec says 404)
SEM-3	user	Medium	DELETE /user/{username} second call is non-deterministic — sometimes returns 200 (NOT idempotent, the buggy outcome), sometimes returns 404 (correct)
DATA-1	store	Critical	GET /store/inventory returns case-insensitive duplicate keys (sold and SOLD) and dozens of garbage status values reflecting that status is never validated against the documented enum
DATA-2	pet	High	POST /pet with empty body returns id 9223372036854775807 (Long.MAX_VALUE), so every empty-body POST collides on the same id
DATA-3	global	Low	Several 404 responses return an empty body with no Content-Type header, while other 404s return JSON
DATA-4	user	High	POST /user returns the new user id encoded as a string in the message field of a generic ApiResponse (spec says no body)

Critical defects

AUTH-1: DELETE /pet/{id} does not enforce the api_key header

- Spec: DELETE /pet/{petId} documents api_key as a required header parameter.
- Steps to reproduce:
 1. POST /pet to create a pet with id N.
 2. DELETE /pet/N with no api_key header.
 3. POST /pet again, then DELETE /pet/N with api_key: bogus-value.
- Observed: both DELETES return 200 with body {"code":200,"type":"unknown","message":"<id>"}.

- **Expected:** at minimum a `401 Unauthorized` for the missing-header case.
- **Severity:** Critical (auth bypass).
- **Captured probes:** `04-cross-cutting.json` → `DELETE /pet/{id}` (no `api_key` header) and `DELETE /pet/{id}` (`api_key=bogus-value`).

AUTH-2: `GET /user/login` accepts any credentials

- **Spec:** documents the operation as "Logs user into the system", with `username` and `password` as required query parameters and a `400` error for invalid credentials.
- **Steps to reproduce (four variants):**
 1. `GET /user/login?username=existing-user&password=correct`
 2. `GET /user/login?username=existing-user&password=WRONG`
 3. `GET /user/login?username=does-not-exist&password=anything`
 4. `GET /user/login` (no parameters at all)
- **Observed:** all four return 200 with body `{"code":200,"type":"unknown","message":"logged in user session:<timestamp>"}`. The `<timestamp>` differs per call but the `code`, `type`, and the literal text `logged in user session:` are identical.
- **Expected:** invalid credentials and missing parameters should return 400. Login should differentiate authenticated vs unauthenticated state.
- **Severity:** Critical (no authentication; the endpoint is decorative).
- **Captured probes:** `03-user.json` → `GET /user/login ( ... )` (4 probes).

DISCLOSURE-1: `GET /pet/{non-numeric-id}` leaks an internal exception

- **Steps to reproduce:** `GET /pet/this-is-not-a-number`.
 - **Observed:** 404 with body `{"code":404,"type":"unknown","message":"java.lang.NumberFormatException: For input string: \"this-is-not-a-number\"}"}`.
 - **Expected:** generic error message; no Java exception class names or input echoes.
 - **Severity:** Critical (information disclosure: implementation language, framework details, raw input echo).
 - **Captured probes:** `04-cross-cutting.json` → `GET /pet/this-is-not-a-number` (content-type sniff).
-

High-severity defects

VAL-1: POST /pet accepts an empty body

- Steps to reproduce: POST /pet with body {} (or no body at all).
- Observed: 200 with body {"id":9223372036854775807,"photoUrls":[],"tags":[]}. The id is Java's Long.MAX_VALUE.
- Expected: 405 (or 400) when required fields name and photoUrls are missing.
- Severity: High. Compounds with DATA-2 — every empty-body POST collides on 9223372036854775807.
- Captured probes: 01-pet.json → POST /pet (empty body) and POST /pet (missing required fields).

VAL-2: POST /pet with malformed JSON returns 500 "something bad happened"

- Steps to reproduce: POST /pet with Content-Type: application/json and body this is not json.
- Observed: 500 with body {"code":500,"type":"unknown","message":"something bad happened"}.
- Expected: 400 with a meaningful parse-error message.
- Severity: High. The literal string "something bad happened" is a deliberately bad demo error message and is the canonical "5xx is the server's fault, not the client's" anti-pattern.
- Captured probes: 01-pet.json → POST /pet (malformed body, string).

VAL-3 / VAL-4: GET /pet/findByStatus does not validate the status query parameter

- VAL-3 — invalid status: GET /pet/findByStatus?status=banana returns 200 with body []. Spec says 400.
- VAL-4 — missing required param: GET /pet/findByStatus (no parameter) returns 200 with body []. Spec marks status as required: true and documents 400.
- Expected: both should be 400.
- Severity: High. Combined with DATA-1 this is the root cause of the polluted /store/inventory keys.
- Captured probes: 01-pet.json → GET /pet/findByStatus?status=banana (invalid) and GET /pet/findByStatus (no param).

VAL-5: GET /pet/findByTags does not validate the tags query parameter

- Steps to reproduce: GET /pet/findByTags (no parameter).

- **Observed:** 200 with empty array.
- **Expected:** 400 (spec marks `tags` as required) — the endpoint is also marked deprecated, but that does not justify silent acceptance of an absent required param.
- **Severity:** High.
- **Captured probes:** `01-pet.json` → `GET /pet/findByTags (no param)`.

VAL-6: `POST /store/order` accepts garbage input

- **Steps to reproduce (three variants):**
 1. Empty body: `POST /store/order` with `{}`.
 2. Invalid status: `POST /store/order` with `{ ..., "status": "banana" }`.
 3. Out-of-range id: `POST /store/order` with `{ "id": 9_877_268_767, ... }` (the spec documents `id: 1..10`).
- **Observed:** all three return 200. The out-of-range id is actually persisted — `GET /store/order/9877268767` afterwards returns 200 with the order body, contradicting the spec's stated id range.
- **Expected:** 400 for all three.
- **Severity:** High.
- **Captured probes:** `02-store.json` → `POST /store/order (empty body)`, `POST /store/order (status=banana)`, `POST /store/order id=9877268767 (above range)`, `GET /store/order/9877268767 (verify out-of-range)`.

VAL-7: `POST /user` accepts empty body, duplicates, and special-character usernames

- **Empty body:** `POST /user` with `{}` returns 200 and body `{"code":200,"type":"unknown","message":"0"}`. The `message` field is the new user id, here `0` (which is also a colliding sentinel value for any other empty-body POST).
- **Duplicate username:** posting twice with the same username succeeds both times, with the second call returning a different `message` id. The original record is silently overwritten.
- **Special chars:** `POST /user` with `username = "diag user/with*chars!1777172137731"` returns 200 and the user is later retrievable via `GET /user/{encoded}` with `firstName/lastName/email` intact — meaning the API persists slashes, asterisks, exclamations, and spaces as username characters with no normalisation.
- **Expected:** validation rejecting empty body, duplicates, and reserved characters.
- **Severity:** High.
- **Captured probes:** `03-user.json` → `POST /user (empty body)`, `POST /user (duplicate username)`, `POST /user (special chars in username)`, `GET /user/{specialUsername} (encoded)`.

SEM-1: PUT /pet silently upserts non-existent ids

- Steps to reproduce:
 1. Pick an id that has never been used (9_876_905_853).
 2. PUT /pet with that id and a full body.
 3. GET /pet/9876905853 .
- Observed: PUT returns 200; the GET also returns 200 with the body that was PUT.
- Expected per spec: PUT documents 404 (Pet not found) when the id is unknown. The actual behaviour is upsert.
- Severity: High (semantic deviation from spec).
- Captured probes: 01-pet.json → PUT /pet (upsert orphan id ...) and GET /pet/{orphan} (verify upsert) .

SEM-2: PUT /user/{username} silently upserts non-existent users

- Steps to reproduce:
 1. PUT /user/some-username-that-was-never-created with a full user body.
 2. GET /user/some-username-that-was-never-created .
- Observed: PUT returns 200; the GET returns 200 with the body that was PUT.
- Expected per spec: 404.
- Severity: High.
- Captured probes: 03-user.json → PUT /user/ ... (upsert orphan) and GET /user/ ... (verify upsert) .

DATA-1: GET /store/inventory returns garbage / case-duplicate keys

- Steps to reproduce: GET /store/inventory .
- Observed (single sample run):

```
{
  "sold": 303,
  "AAA": 4,
  "string": 171,
  "NOT available": 1,
  "sold ": 1,
  "d": 1,
  "Busy": 1,
  "pending": 59,
  "available": 409,
  "weisskeiner1": 1,
  "avalible": 1,
```

```

    "False": 2,
    "availabl": 1,
    "0": 2,
    "SOLD": 1,
    "available, sold": 1,
    "avaliable": 1,
    "8": 1
  }

```

- **Findings in this body:**
 - `sold` and `SOLD` are case-only duplicates that JSON-strict consumers (e.g., PowerShell `ConvertFrom-Json`) refuse to parse.
 - `sold` (with a trailing space) is a third near-duplicate.
 - `avaliabl`, `availabl`, `avaliable` are typo variants of `available`.
 - `False`, `0`, `8`, `Busy`, `string`, `AAA`, `NOT available`, `available`, `sold`, `weisskeiner1`, `d` are entirely outside the documented enum (`available` | `pending` | `sold`).
- **Expected:** the documented enum should be enforced at write time; `inventory` should only ever return three keys.
- **Severity:** Critical (data integrity + downstream parser breakage). Root cause is **VAL-3 / VAL-4** — `findByStatus` and `POST /pet` never validate `status` against the enum.
- **Captured probes:** `02-store.json` → `GET /store/inventory`.

DATA-2: Empty-body `POST /pet` collides on id `9223372036854775807`

- Direct consequence of **VAL-1**. The server assigns `Long.MAX_VALUE` to the `id` field of an empty-body `pet`, so any two empty-body `POSTs` from any clients on Earth share the same key.
- **Severity:** High.

DATA-4: `POST /user` returns the new user id in the `message` field

- **Steps to reproduce:** `POST /user` with a valid body. Read the response.
 - **Observed:** body `{"code":200,"type":"unknown","message":"3519290"}`. The numeric string in `message` is the auto-assigned user id; the spec documents no response body for the success case.
 - **Expected:** either return the full user object (REST-style), or return `Location: /v2/user/<id>`. Encoding the id as a string in `message` of a generic `ApiResponse` envelope forces clients to do magic-string parsing.
 - **Severity:** High (API design defect; spec deviation).
 - **Captured probes:** `03-user.json` → `POST /user` (happy), `POST /user` (special chars in username) (id surfaces as `"message":"3519290"`).
-

Medium-severity defects

CODE-1 / CODE-2 / CODE-3: 4xx status codes flattened to 404

The spec distinguishes `400 Invalid ID supplied` from `404 Pet not found` (and equivalents for orders / users). The API does not.

Probe	Observed	Expected per spec
GET /pet/0	404	400
GET /pet/-1	404	400
GET /pet/not-a-number	404 (with leaked stack trace, see DISCLOSURE-1)	400
GET /store/order/-1	404	400
GET /store/order/abc	404	400
DELETE /store/order/abc	404	400

- **Severity:** Medium (semantic information loss; clients cannot distinguish "your input is structurally bad" from "the resource does not exist").
- **Captured probes:** `01-pet.json` and `02-store.json`.

SEM-3: DELETE /user/{username} second call is non-deterministic

- **Steps to reproduce:**
 1. Create a user.
 2. `DELETE /user/{username}` — always returns 200.
 3. `DELETE /user/{username}` again — observe response.
- **Observed:** the second call is **non-deterministic**. The first diagnostic run returned `200` with body `{"code":200,"type":"unknown","message":"<username>"}` (the buggy outcome — not idempotent). A subsequent run of the regression test returned `404` (the correct REST outcome). The inconsistency itself is the defect — clients cannot rely on the contract.
- **Expected:** the second call should reliably return 404 (the resource no longer exists). Compare with `DELETE /pet/{id}` which **does** correctly and reliably transition `200` → `404` on the second call.
- **Severity:** Medium.
- **Captured probes:** `03-user.json` → `DELETE /user/...` (**idempotent**) initially captured `200`. The regression test `SEM-3: DELETE /user/{username} second call is non-deterministic` accepts either outcome and pins the envelope.

VAL-8: Batch user-creation accepts empty arrays

- Steps to reproduce: `POST /user/createWithArray` with body `[]` .
- Observed: 200 with body `{"code":200,"type":"unknown","message":"ok"}` .
- Expected: 400 (no users to create).
- Severity: Medium.

VAL-9: `POST /pet/{id}` form-data accepts an empty form

- Steps to reproduce: `POST /pet/{existing-id}` with no `name` and no `status` form fields.
 - Observed: 200.
 - Expected: 405 (per spec) or 400.
 - Severity: Medium.
-

Low-severity defects

DATA-3: Some 404 responses lack a body and `Content-Type` header

- Affected probes (single sample):
 - `DELETE /pet/{id}` (idempotent: second call) → 404, body empty, no `Content-Type` .
 - `DELETE /pet/{nonexistent}` → 404, body empty, no `Content-Type` .
 - `DELETE /user/never-existed` → 404, body empty, no `Content-Type` .
 - Compare to `GET /pet/{nonexistent}` → 404 with proper JSON body and `Content-Type: application/json` .
 - Expected: consistent error envelope across endpoints.
 - Severity: Low (cosmetic / consistency).
-

Things that work correctly (worth noting)

The diagnostic also confirmed a number of behaviours that *are* correct or non-buggy. Listed here so the catalogue is balanced and so future regressions can be detected.

- Schema fidelity on `POST /pet` → `GET /pet/{id}` round-trip is exact. The full body (including `category` , `tags` , `photoUrls`) round-trips byte-for-byte. Cross-cutting probe `Schema fidelity: POST /pet vs GET round-trip` .

- **Eventual consistency is *not* an issue at the moment.** GET immediately after POST returned 200 on every retry-loop probe (pet, order, user) on the first attempt. Petstore's reputation for consistency lag did not reproduce in this run.
 - **DELETE /pet/{id} idempotency is correct (SEM-4).** First call 200, second call 404. (Contrast with SEM-3 on user deletion.)
 - **DELETE /store/order/{id} idempotency is correct.** First call 200, second call 404.
 - **Wrong-content-type rejection works.** POST /pet with Content-Type: text/plain returns 415 (Unsupported Media Type) — proper behaviour.
 - **OPTIONS /pet returns 204** with no Access-Control-Allow-* headers from a third-party origin — the API is therefore *not* CORS-enabled for browser clients on other origins (this is by design for a server-to-server API).
 - **GET /user/{nonexistent} returns 404 with a clean JSON error body**
({ "code":1, "type":"error", "message":"User not found" }).
-

Investigation gaps

The diagnostic ran 78 probes; the public sandbox has more behaviour surface than that. Out of scope of this run:

- **POST /pet/{id}/uploadImage** — the diagnostic only checked status code (200). Whether the image bytes are actually persisted, retrievable, or referenced by photoUrls was not verified.
- **Concurrency / race conditions** — concurrent POST /pet with the same id, concurrent DELETE + GET , etc.
- **Very large payloads** — name longer than X characters, very large photoUrls arrays, deeply nested categories.
- **Numeric overflow** — sending an id larger than Long.MAX_VALUE , sending a negative id on POST.
- **Header injection** — values for api_key containing CRLF, very long header values.
- **Rate limiting / throttling** — number of requests per second the sandbox tolerates before degrading.
- **Pagination** — findByStatus returns hundreds of pets in a single response; the API documents no limit / offset . Behaviour at scale not probed.
- **GET /user/login session semantics** — whether the session timestamp is honoured anywhere, whether /user/logout invalidates anything.
- **Schema fidelity on Order and User** — verified for Pet only.
- **Inventory cleanup over time** — whether the garbage status keys (AAA , Busy , etc.) are eventually pruned or whether they grow unboundedly.